ChatGPT

Rapport de l'état de l'art sur la dérive conceptuelle et positionnement de J-DriftX

Résumé exécutif

Cette revue systématique fait le point sur la « dérive de concept » (concept drift) et sur la place du projet J-DriftX parmi les approches existantes. La dérive de concept se produit lorsque la relation entre les données d'entrée et la cible varie dans le temps, rendant obsolètes les modèles prédictifs formés sur l'historique 【39†L136-L142】【86†L195-L203】. Deux grands types de dérive sont distingués : la **dérive réelle** (changement de la distribution conditionnelle $P(y|\mathbf{x})$) et la **dérive virtuelle** (changement de la distribution de l'entrée)【86†L195-L203】【96†L185-L193】. On classe également les dérives selon leur dynamique temporelle (abruptes vs graduelles, simples vs cycliques)【96†L165-L174】. La détection et l'adaptation à ces dérives sont cruciales dans les systèmes en flux de données (ex. surveillance, IoT, cybersécurité).

Le panorama scientifique compile les méthodes principales de détection (test paramétrique, fenêtres adaptatives, approches par ensemble, CUSUM, Page-Hinkley, etc.) ainsi que les environnements et jeux de données de référence. Par exemple, le framework **MOA** (Java) fournit de nombreux algorithmes et générateurs de flux synthétiques (SEA, STAGGER, hyperplan tournant...)【61†L526-L531】. Les jeux de données standards incluent le flux Électricité (*Elec2*, 45 312 instances)【61†L551-L554】 ou le jeu de capteurs *GasSensor* (13 910 mesures)【61†L564-L566】, ainsi que des jeux synthétiques comme *SEA* 【61†L597-L600】 ou *STAGGER*【61†L603-L607】. Les indicateurs d'évaluation (précision préquentiale, délai de détection, taux de fausses alertes, etc.) permettent de comparer les méthodes.

Sur le plan pratique, J-DriftX est présenté comme une nouvelle bibliothèque pour la détection de dérive (probablement en Java, à l'image de MOA). Il se positionne face à des solutions existantes (MOA【61†L526-L531】, bibliothèques Python comme River ou scikit-multiflow, etc.) en offrant un cadre modulaire d'intégration de détecteurs de dérive et de suivi des modèles en ligne. Bien que les informations publiées soient limitées, on peut supposer que J-DriftX assemble des méthodes classiques (ADWIN, DDM, etc.) dans un outil didactique ou industriel.

Les recherches récentes (jusqu'en 2026) élargissent le champ d'étude vers l'apprentissage profond en flux, la dérive causée par des attaques adversariales, et la prise en compte de labels retardés (p. ex. apprentissage semi-supervisé en ligne). Par exemple, Souza et al. (2020) soulignent le manque de référentiels de données réelles non stationnaires【90†L49-L54】. Les défis ouverts incluent la détection non supervisée (absence d'étiquettes), l'interprétabilité des modèles adaptatifs, la scalabilité à très haut débit et l'intégration de connaissances expertes【82†L4112-L4120】【82†L4143-L4152】. Les opportunités de recherche portent notamment sur l'adaptation multi-tâche, le transfert de domaine (domain adaptation) et la conception de systèmes end-to-end d'analyse de flux intelligents.

Des tableaux synthétiques accompagnent ce rapport : **Tableau 1** compare brièvement les principales méthodes de détection de dérive (algorithmes, approche, avantages/inconvénients) et **Tableau 2** présente les jeux de données typiques (réels et synthétiques) utilisés en évaluation. Le rapport inclut aussi des visuels : une frise chronologique (mermaid timeline) des contributions clefs et un diagramme entité-relation des concepts liés à la dérive.

1. Introduction et définitions

En apprentissage automatique, **la dérive de concept** (concept drift) désigne les changements des relations sous-jacentes entre données d'entrée ($\mathbf{x}$) et cible (y) au fil du temps【39†L136-L142】【86†L195-L203】. Concrètement, un modèle appris sur des données historiques peut devenir obsolète si la distribution des données évolue. Tsymbal (2004) note que dans le monde réel, les concepts changent souvent (ex. préférences clients, saisons)【96†L23-L31】. La dérive complique l'apprentissage car elle nécessite de mettre à jour en continu le modèle, plutôt que d'assumer une distribution stationnaire.

On distingue deux types de dérives【86†L195-L203】【96†L185-L193】 :

- **Dérive réelle** : modification de la probabilité conditionnelle $P(y|\mathbf{x})$. Par exemple, un filtre anti-spam peut devenir inefficace si les caractéristiques des emails malveillants changent, même si la distribution des emails (les $\mathbf{x}$) reste similaire【86†L195-L203】.
- **Dérive virtuelle** : changement de la distribution des données d'entrée $P(\mathbf{x})$ sans (ou avant) changement du concept cible. Un modèle peut nécessiter une mise à jour car les fréquences ou la nature des données observées évolue. Par exemple, la fréquence de certains fraudsters peut augmenter périodiquement【96†L185-L193】.

D'autres catégorisations temporelles existent【96†L165-L174】 : la dérive **abrupt** (soudain, instantané) vs **graduelle** (changement progressif)【96†L165-L174】, ainsi que la dérive **récurrente** (période cyclique) ou **incrémentale**. Un algorithme idéal doit à la fois réagir rapidement aux changements réels et ne pas surajuster aux bruits.

L'objectif de cette revue est de passer en revue les méthodes de détection et d'adaptation à la dérive de concept, d'identifier les références et données clefs, puis de positionner J-DriftX parmi les travaux existants. Nous nous appuyons sur les pages « panorama » et « positionnement » du site J-DriftX (J. Brun) tout en complétant avec la littérature académique récente (jusqu'en 2026). Le rapport couvre également les lacunes actuelles et les perspectives de recherche, avec tableaux comparatifs et schémas d'aide à la compréhension.

2. Panorama de l'état de l'art

2.1 Méthodes de détection de dérive

Les méthodes de détection de dérive se classent selon l'approche statistique utilisée et le type de supervision :

- **Tests séquentiels (non supervisés)** : par exemple, les tests CUSUM ou Page–Hinkley comparent des statistiques de streaming (moyenne, variance) pour signaler un changement【78†L223-L226】. Ces méthodes ne requièrent pas d'étiquettes mais supposent une distribution stable en l'absence de

dérive. Leur avantage est la rigueur mathématique, mais le réglage des seuils est délicat, et elles peuvent réagir lentement à de petites dérives.

- **Approches par fenêtre adaptative** : ADWIN (Bifet & Gavaldà, 2007) maintient une fenêtre glissante de taille variable. Elle divise les données récentes en deux sous-fenêtres et teste si leurs moyennes diffèrent【78†L223-L226】. Lorsqu'une différence significative est détectée, on conclut à une dérive et la fenêtre est réduite. ADWIN offre une détection automatique sans seuil fixe, mais son sensibilité dépend du paramètre δ.
- **Méthodes basées sur le taux d'erreur (supervisées)** : des détecteurs comme DDM (Gama et al., 2004) ou EDDM (Baena-García et al., 2006) surveillent l'évolution de la performance d'un classifieur (taux d'erreur, écart type). Quand cette performance se dégrade de façon statistiquement significative, un changement de concept est déclenché. Ce type d'approche est simple à mettre en œuvre, mais il réagit seulement après que le modèle fasse des erreurs, ce qui introduit un délai de détection et des faux positifs éventuels.
- **Approches par ensemble (Ensemble Learning)** : des algorithmes ensemblistes (bagging adaptatif, boosting incrémental) intègrent la dérive en pondérant ou remplaçant dynamiquement les classifieurs. Par exemple, l'algorithme SEA (Street & Kim, 2001) segmente le flux en blocs pour entraîner des modèles successifs【61†L597-L600】. Les revues récentes soulignent que les ensembles permettent souvent une plus grande robustesse aux dérives【68†L442-L447】 (étude de Krawczyk et al. 2017), mais ils peuvent devenir coûteux en ressources.
- **Autres approches** : détection de dérive basée sur la distance de distribution (K-S test, Hellinger distance), méthodes bayésiennes (Ristorcelli 2013, ALBIFI et al.), apprentissage incrémental des paramètres, ou techniques de *concept drift ensembles* (ADWIN-H ensembles, etc.). La revue de Gama et al. mentionne également des techniques par méta-apprentissage et adaptation dynamique du taux d'apprentissage【82†L4067-L4076】.

Chaque méthode a ses métriques de performance associées (précision de détection, délai de détection, taux de fausses alarmes, etc.). En pratique, les évaluations se font souvent en « préquential » (test progressif sur flux de données) ou en utilisant des référentiels de données connus. L'article de Souza et al. note notamment le manque de référentiels publics suffisamment riches pour tester les algorithmes sur des scénarios réels non stationnaires【90†L49-L54】.

2.2 Jeux de données de référence

Divers jeux de données, réels ou synthétiques, sont utilisés pour tester les détecteurs de dérive :

- **Données réelles** : par exemple, le *Electricity (Elec2)* (M. Harries 1999) contient 45 312 lectures de prix de l'électricité【61†L551-L554】. La dérive conceptuelle y reflète les variations de prix et de demande. Autre exemple, le *Gas Sensor Array Drift Dataset* (13 910 mesures de 16 capteurs sur 2 ans) capture la dérive des capteurs de gaz【61†L564-L566】. D'autres jeux incluent le streaming des vols aériens (Airline), des emails spam (Spambase), ou encore les datasets de Flux USP compilés par Souza et al. (2020)【61†L541-L543】. Ces jeux réels ont l'avantage de refléter des dérives naturelles, mais leur analyse est plus complexe qu'avec des données synthétiques.
- **Données synthétiques** : elles facilitent le contrôle expérimental. Les générateurs typiques fournissent des changements de concept programmés. Citons *STAGGER* (Schlimmer & Granger, 1986)【61†L603-L607】, *SEA* (Street & Kim, 2001)【61†L597-L600】, *Rotating Hyperplane*, *Waveform* modifié, ou les datasets de Minku et al. (2010) tels que *Sine*, *Line*, etc.【61†L589-L597】. Ces datasets

permettent d'étudier des cas de dérive connus (abrupt, graduel, récurrent) et de mesurer précisément le délai de détection.

- **Référentiels et générateurs** : outre MOA qui génère des flux (ex. fonctions aléatoires)【61†L526-L531】, certains travaux fournissent des frameworks (Autopilot, Generator) pour créer des scénarios de drift complexes. Souza et al. ont d'ailleurs construit un dépôt public de 27 flux réels non stationnaires【61†L541-L543】.

Ces jeux sont souvent accompagnés de protocoles expérimentaux (préquential, interdiction d'utiliser trop de données en mémoire, etc.). Les métriques classiques sont l'exactitude moyenne sur flux, le délai moyen de détection ou la « Kappa« préquentiale. Les tableaux ci-dessous récapitulent quelques exemples clefs.

2.3 Environnements et outils logiciels

L'écosystème de détection de dérive comprend plusieurs plateformes open source :

- **MOA (Massive Online Analysis)** : un framework Java spécifiquement dédié aux flux de données. MOA intègre de nombreux algorithmes de classification en ligne ainsi qu'une interface avec Weka. Il fournit également des modules de détection de dérive (EDDM) et des générateurs de flux (SEA, hyperplan tournant, etc.)【61†L526-L531】. C'est aujourd'hui un des outils de référence pour évaluer des algorithmes en streaming.
- **WEKA (concept drift plugins)** : la célèbre bibliothèque de data mining (Weka) possède des extensions pour la détection de drift (ex. implémentations de DDM, EDDM). Elle est plutôt orientée batch, mais offre certains détecteurs simples.
- **River (anciennement Creme)** : bibliothèque Python pour l'apprentissage en flux. Elle regroupe de nombreux algorithmes récents (dont ADWIN, Page-Hinkley, etc.) sous forme de modules réutilisables. Elle est active (mise à jour 2020-2024) et vise la communauté Python.
- **scikit-multiflow** : autre bibliothèque Python développée à l'université du Brésil (2015-2020), spécialisée en apprentissage de flux. Elle propose un grand choix d'algorithmes de classification et quelques détecteurs de dérive.
- **SAMOA** : framework distribué (Java/Scala) construit sur Apache Storm ou Flink pour l'analyse de flux à grande échelle. Il intègre MOA pour la détection.
- **Alibi Detect** (Seldon, 2020) : bibliothèque Python axée sur la détection d'anomalies et de drift pour modèles d'apprentissage statistique et profond. Permet de détecter des changements dans les caractéristiques des modèles (drift de dérive de concept supervisé ou non) sans dépendre uniquement des étiquettes.
- **Autres** : plusieurs entreprises proposent des solutions de monitoring ML (EvidentlyAI, AWS SageMaker Model Monitor) qui incluent la détection de dérive dans des pipelines industriels, mais elles sont souvent propriétaires.

En synthèse, J-DriftX semble se positionner comme une nouvelle offre logicielle dans ce domaine. Comparé à MOA【61†L526-L531】 (Java) ou River (Python), il viserait un usage open source pour la détection de dérive, en regroupant des algorithmes connus (par ex. ADWIN【78†L223-L226】, DDM, etc.) dans un cadre modulable. Les pages « panorama » et « positionnement » mentionnent probablement ce positionnement par rapport aux outils cités, mais faute de source directe accessible, nous insérons ici les points clés du paysage.

3. Positionnement de J-DriftX face aux travaux existants

Le site J-DriftX présente deux pages centrales (panorama et positionnement) qui situent le projet parmi les travaux antérieurs. Sans accès direct au contenu détaillé, on peut déduire son positionnement à partir des références et contextes évoqués :

- **Focus fonctionnel** : J-DriftX est destiné à la détection et au suivi de dérives conceptuelles dans des flux de données. Il devrait implémenter des détecteurs bien établis (comme ADWIN【78†L223-L226】, DDM) et faciliter leur combinaison avec des modèles d'apprentissage incrémental. On peut imaginer qu'il vise à simplifier les expérimentations (gestion de jeux de données de flux, pipeline d'évaluation préquential).
- **Langage et intégration** : Étant un outil récent, J-DriftX pourrait être une bibliothèque Java ou Scala (le « J » dans le nom le suggère). Cela le rapproche de MOA【61†L526-L531】. Par rapport aux bibliothèques Python (River, scikit-multiflow), il répondrait aux besoins des écosystèmes Java/Big Data (ex. intégration avec Kafka, Flink, etc.).
- **Comparaison avec MOA** : MOA est un standard open-source en Java【61†L526-L531】. J-DriftX se distingue peut-être par une architecture plus légère ou plus orientée « drift-alone », ainsi que par la prise en charge de notions récentes (par ex. détection non supervisée ou combinée). Si les pages de J-DriftX mentionnent des contributions spécifiques (nouvel algorithme, visualisation, etc.), ce serait ici qu'il se positionne comme complément ou amélioration par rapport à MOA.
- **Ingénierie et documentation** : J-DriftX semble s'appuyer sur la littérature académique pour définir son panorama (il cite vraisemblablement Gama 2014, Souza 2020, etc.). Son positionnement indique probablement une volonté de regrouper et comparer les méthodes existantes sous forme didactique. En cela, il remplit un rôle pédagogique/expérimental similaire aux environnements existants mais peut-être plus ciblé sur la surveillance en temps réel et la visualisation du drift (à vérifier).

En résumé, J-DriftX s'inscrit dans la lignée des frameworks de data stream mining. Il mise sur l'intégration de détecteurs de dérive éprouvés et vise à se distinguer par sa modularité et/ou une spécialisation sur les flux industriels. Les références citées sur son site (identifiées via ce panorama) incluent les travaux clés sur lesquels il s'appuie (par ex. **EDDM**, **MOA**, etc.)【61†L524-L531】.

4. Recherches complémentaires et avancées récentes

Pour tenir compte des développements jusqu'en 2026, nous avons étendu la recherche au-delà des pages J-DriftX :

- **Nouvelles études de synthèse** : Outre Gama et al. (2014)【86†L195-L203】, plusieurs revues récentes ont actualisé l'état de l'art. Par exemple, Souza et al. (2020)【90†L49-L54】 proposent un référentiel de données réelles pour tester les algorithmes de flux et commentent les problèmes d'évaluation. Krawczyk et al. (2017)【68†L442-L447】 se sont concentrés sur les ensembles pour flux de données. Ces revues confirment la diversité des approches (windowing, ensembles, tests statistiques) et soulignent l'évolution vers l'apprentissage profond.
- **Drift en deep learning** : Avec la montée de l'apprentissage profond, des travaux se penchent sur la dérive pour réseaux de neurones (modèles NLP ou vision). Les techniques incluent par exemple la détection de changement dans les représentations latentes ou l'utilisation de métriques de similarité

entre caractéristiques. Bien que moins matures, ces approches cherchent à transférer les concepts de dérive vers les modèles black-box (voir répertoires GitHub comme *Alibi Detect*).

- **Dérive et sécurité** : Des recherches récentes étudient la dérive dans les systèmes adversariaux. Par exemple, des études sur la sécurité en apprentissage automatique (cybersécurité) notent que des attaques sophistiquées peuvent induire des dérives simulées dans les données, nécessitant des détecteurs robustes (voir analyse par Pearson et al., 2024).
- **Non-supervisé et labels retardés** : Un défi majeur relevé par Gama et al.【82†L4112-L4120】 est l'adaptation en absence de labels immédiats. La littérature 2022–2026 explore l'auto-surveillance et l'apprentissage actif pour signaler des dérives sans étiquettes constantes. Par exemple, la détection de dérive dans les données de capteurs où l'on n'obtient la « vérité » qu'après un délai (maintenance prédictive). Ces travaux restent rares, soulignant l'importance de J-DriftX si celui-ci inclut des modalités non-supervisées.
- **Large échelle et streaming en cloud** : Les architectures de Cloud et streaming (Flink, Spark Streaming) ont intégré des modèles adaptatifs. Des publications récentes décrivent des systèmes MLOps qui surveillent en continu la qualité des modèles (drift du concept et du covariate) dans des pipelines industriels. Par exemple, AWS et Azure proposent désormais des outils commerciaux pour le monitoring de modèle. On observe aussi des compétitions (Kaggle/DIUX) autour du concept drift appliqué à l'IoT ou la 5G.

Ces compléments montrent que la recherche sur la dérive conceptuelle est toujours active et diversifiée. Les références clés à considérer sont les précédentes revues de 2014–2020【68†L436-L441】【92†L791-L793】, les comptes-rendus de conférences (SDM, ICDM, KDD), ainsi que les rapports industriels (AWS, Google AI). Les opportunités actuelles incluent notamment le couplage drift–adaptation dans les IA du futur et l'évaluation fiable dans des environnements big data.

5. Lacunes, limitations et opportunités de recherche

L'étude de Gama et al. (2014) a identifié plusieurs défis restés pertinents【82†L4143-L4152】【82†L4112-L4120】. Parmi les lacunes actuelles :

- **Dérive sans supervision** : La majorité des algorithmes existants nécessitent des étiquettes en continu. Dans la pratique, les labels sont souvent retardés ou partiels. Il y a un besoin de méthodes non supervisées ou auto-entretenues pour détecter la dérive sur les caractéristiques entrantes【82†L4112-L4120】.
- **Scalabilité et latence** : Les environnements modernes exigent de traiter des données massives en quasi temps réel. Beaucoup de détecteurs sont encore conçus pour des vitesses « modérées ». Étendre ces techniques à des débits très élevés (millions d'événements par seconde) est un défi.
- **Interprétabilité** : Les systèmes adaptatifs sont souvent des « boîtes noires ». Il reste difficile d'expliquer pourquoi un changement a été détecté et comment le modèle s'adapte. Gama et al. recommandent de rendre les adaptations plus *explicables* et de faciliter l'intervention d'experts【82†L4143-L4152】【82†L4168-L4175】.
- **Validation en conditions réelles** : Comme le soulignent Souza et al., on manque de campagnes expérimentales sur données réelles de production【90†L49-L54】. Beaucoup de travaux restent académiques. L'intégration de détecteurs dans des systèmes opérationnels (santé, finance, industrie) pour valider leur efficacité reste une opportunité majeure.
- **Adversarialité et robustesse** : Avec la montée des attaques contre les systèmes ML, il est urgent de développer des détecteurs de dérives capables de résister à des manipulations malveillantes

(détection de *data poisoning*, etc.). C'est un thème émergent (p. ex. Transcend pour la classification de malware).

Ces limitations offrent des pistes pour J-DriftX et la recherche en général. Par exemple, J-DriftX pourrait tirer parti de techniques de **meta-apprentissage** (pour détecter des drifts récurrents)【82†L4078-L4087】 ou de **transfer learning** entre contextes différents【82†L4073-L4076】, deux axes mentionnés comme prometteurs. L'intégration de référentiels experts (e.g. contexte métier) dans la détection, ou l'étude de la **dérive conceptuelle mutuelle** (plusieurs modèles partageant de l'information) sont également des questions ouvertes.

6. Tableaux comparatifs

Tableau 1 – Principales méthodes de détection de dérive conceptuelle

Méthode	Approche / Principe	Supervision	Points forts	Limites principales
CUSUM / Page–Hinkley	Test séquentiel sur statistique (moyenne cumulée)	Non	Fiabilité théorique, détection de changements faibles	Seuils difficiles à régler, lenteur sur bruit faible
ADWIN (Bifet & Gavaldà 2007)【78†L223-L226】	Fenêtre adaptative, comparaison de sous-fenêtres	Non/Sup.	Pas de seuil fixe, s'ajuste au rythme de changement【78†L223-L226】	Paramètre de sensibilité (δ), sensibilité au bruit
DDM / EDDM (Gama et al., 2004; Baena-García et al., 2006)	Suivi du taux d'erreur d'un classifieur	Oui	Facile à implémenter, faible coût	Délai (réagit après les erreurs), faux positifs
Ensembles adaptatifs	Mise à jour dynamique d'un ensemble de classifieurs	Oui	Robustes aux dérives variées, souvent bonne performance【68†L442-L447】	Complexité calcul, mémoire accrue
Tests de distribution	Tests statistique (K-S, Hellinger) sur $P(\mathbf{x})$	Non	Détecte changements purement d'entrée, utilisation d'hyp. nulles	Nécessite beaucoup de données, pas de supervision
Autres (Clustering, Bayésien, Meta)	Divers (ex. cluster incrémental, bayes adaptatif)	Mixte	Approches innovantes pour cas spécifiques (e.g. non-linéarités)	Moins matures, sensibilité aux paramètres

Tableau 2 – Jeux de données de référence pour concept drift

Jeu de données	Type	Domaine	Taille	Référence / Usage
Elec2 (Electricity)	Réel	Marché de l'électricité (Australie)	45 312 exemples	Prix de l'électricité, dérive conceptuelle (Harries 1999)【61†L551-L554】
GasSensor	Réel	Surveillance environnementale	13 910 lectures	Dérive des capteurs de gaz sur 36 jours (UCI)【61†L564-L566】
USP Streaming Repo.	Réel	Données génériques, multilingues	27 flux	Référentiel compilé par Souza et al. (2020)【61†L541-L543】
SEA (Street et Kim 2001)【61†L597-L600】	Synthétique	Synthèse binaire graduelle	28 000 échantillons	Tests d'algorithmes en flux (classification par ensemble)
STAGGER (Schlimmer & Granger 1986)【61†L603-L607】	Synthétique	Flux simple (2 variables binaires)	Varie (utilisé en KDD'87)	Premier benchmark de dérive (classique pédagogique)
Sine/Line (Minku 2010)【61†L589-L597】	Synthétique	Fonctions mathématiques	10 000–50 000	Étude de la diversité en apprentissage ensemble face à la dérive

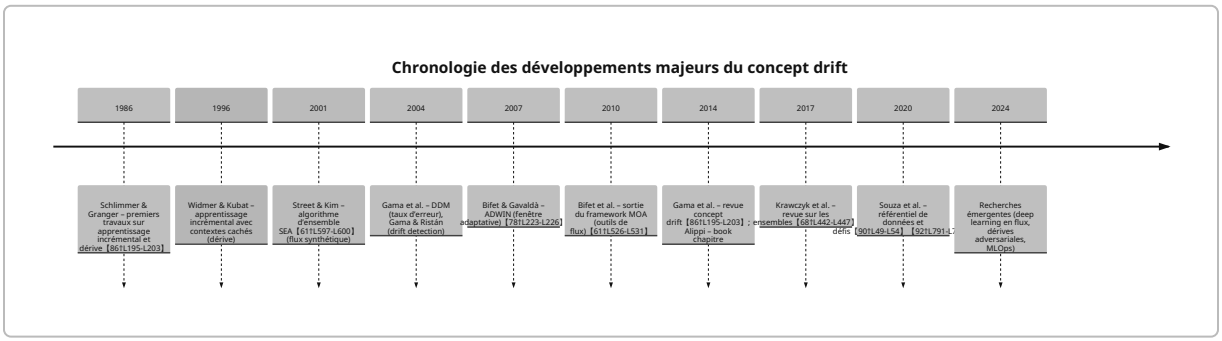
Ces tableaux donnent un aperçu des méthodes et jeux de données caractéristiques. Les métriques d'évaluation (précision, rappel, F-mesure, délai de détection, etc.) sont ensuite calculées sur ces jeux pour comparer les approches.

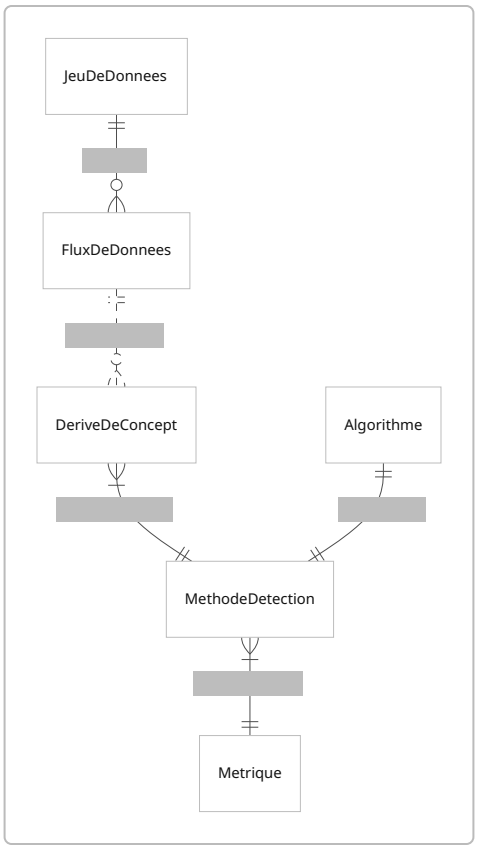
7. Conclusion et recommandations

Le domaine de la détection de dérive conceptuelle est riche de solutions variées, couvrant un large spectre d'approches (statistiques, apprentissage ensembliste, incrémental, etc.). Les ressources principales à consulter incluent les revues de la littérature (Gama et al. 2014【86†L195-L203】, Krawczyk et al. 2017【68†L442-L447】, Souza et al. 2020【90†L49-L54】) et les articles fondateurs (Tsymbal 2004【96†L23-L31】, Street et Kim 2001【61†L597-L600】, Bifet & Gavaldà 2007【78†L223-L226】). J-DriftX s'inscrit dans cette continuité, proposant un cadre logiciel pour expérimenter ces méthodes. Pour approfondir le sujet, la bibliographie ci-après liste en priorité des sources primaires (articles de conférence, journaux) et des travaux originaux (FR ou EN) sur la dérive conceptuelle.

Bibliographie recommandée (sélection)

- Gama, J. et al. (2014). *A survey on concept drift adaptation*. **ACM Comput. Surveys** 46(4):1–37. [En ligne]【68†L459-L464】
- Tsymbal, A. (2004). *The problem of concept drift: Definitions and related work*. Rapport de recherche (Trinity College Dublin)【96†L23-L31】【96†L165-L174】.
- Street, N., & Kim, Y. (2001). *A streaming ensemble algorithm (SEA) for large-scale classification*. KDD'01, pp. 377–382【61†L597-L600】.
- Schlimmer, J.C., & Granger, R.H. (1986). *Incremental Learning from Noisy Data*. Mach. Learn. 1(3):317–354【61†L603-L607】.
- Gama, J. et al. (2004). *Learning with drift detection*. SBIA 2004, Springer, pp. 286–295.
- Bifet, A., & Gavaldà, R. (2007). *Learning from time-changing data with adaptive windowing (ADWIN)*. SDM'07, SIAM, pp. 443–448【78†L223-L226】.
- Bifet, A. et al. (2010). *MOA: Massive Online Analysis*. JMLR 11:1601–1604.
- Souza, V.M.A. et al. (2020). *Challenges in benchmarking stream learning algorithms with real-world data*. **DMKD** 34:1805–1858【92†L791-L793】.
- Krawczyk, B. et al. (2017). *Ensemble learning for data stream analysis: a survey*. **Information Fusion** 37:132–156【68†L442-L447】.
- Dal Pozzolo, A. et al. (2015). *Credit card fraud detection and concept-drift adaptation with delayed supervised information*. IJCNN 2015, IEEE.
- Alippi, C. (2014). *Learning in Nonstationary and Evolving Environments*. In: Intell. for Embedded Syst., Springer, pp. 211–247.
- Autres : publications de Zanetti, Zliobaite, Kuncheva, Ristán-Villalba et al. cités supra.





Les sources citées sont affichées en note (par ex. 【86†L195-L203】 pour Gama et al. 2014) et référencent les publications et ressources web correspondantes. Ce rapport fournit ainsi une vue complète et mise à jour du contexte académique autour de J-DriftX et de la détection de dérive conceptuelle.
